MACHINE LEARNING TASK RELEASED

TASK 1: AUTOMATED BIDDING MODEL FOR USED CAR AUCTIONS

The Scenario

You have been recruited as the lead quantitative analyst for Pawn Stars, an automated bidding firm that participates in large-scale wholesale used-car auctions across the United States. Unlike a human buyer who walks the lot, your firm bids on thousands of vehicles per day—purely from data.

Your mission is twofold:

1. Train a Machine Learning model to predict the sellingprice (the final hammer price) of a used car from its features alone.
2. Write the logic for an autonomous bidding agent that participates in a live auction simulation—evaluating cars, placing bids, and managing a $500,000 budget to turn a profit against rival agents.

The Dataset

You are provided with car_auction_train.csv, containing historical records of used-car auction transactions across the US.

Column	Description
year	Manufacture year of the vehicle
make	Brand / manufacturer (Ford, Toyota, BMW, ...)
model	Specific model name
trim	Variant / trim level (LX, EX, Sport, ...)
body	Body type (Sedan, SUV, Pickup, Coupe, ...)
transmission	Transmission type (automatic / manual)
state	US state where the auction took place
condition	Condition rating on a numeric scale
odometer	Mileage at time of sale
color	Exterior colour
interior	Interior colour
sellingprice	Target variable — final hammer price

Warnings — Read These Carefully

The Agent Pipeline Rule: Your trained model will be loaded inside a live auction simulation. The agent receives one car at a time as a dictionary. Your preprocessing must work on a single row —no df.mean(), no df.groupby() inside analyze_item(). Fit all encoders on the training set, save them, and load them in __init__.

Primary Objectives

1. Data Cleaning & Wrangling

Handle nulls, outliers, malformed entries, and high-cardinality columns. Every decision must be justified —why did you impute vs. drop? What value did you impute with and why? What threshold did you use for outlier removal?

2. Exploratory Data Analysis

Go beyond bar charts and histograms. We want to see analysis that reveals non-obvious pricing patterns, such as:

- How depreciation behaves across body types over the years.
- Identifying condition rating thresholds where price collapses non-linearly.
- Odometer-to-year ratio (usage intensity) effectiveness compared to single features.
- Transmission × body type combinations that outperform segment averages.

3. Model Training & Hyperparameter Tuning

Train a regression model to predict sellingprice. Hyperparameter tuning is mandatory.

- Justification for model choice and validation strategy.
- Methods used (e.g., GridSearchCV, RandomizedSearchCV).
- Comparison of tuned model vs. default parameters on the same validation set.

Deterministic Bid Sequence

Design a mathematical sequence for bidding in place_bid. A deterministic sequence means your next bid is a pure function of:

- self.predicted_value
- self.bankroll
- current_highest_bid
- The auction round number (tracked internally)

Include a "Bid Sequence Design" section in your report covering the formula, theoretical advantages over flat increments, and aggression vs. margin trade-offs.

Submission Format

Submit a single ZIP file: YourName_IronLot.zip.

- agent_YourName.py
- model_YourName.pkl
- encoders_YourName.pkl
- analysis_YourName.ipynb
- report_YourName.pdf

Note: Always use relative paths for loading model files in your script to ensure compatibility in the Arena.

Evaluation Criterion	Weightage
Total Net Profit	15%
Number of Wins	20%
Data Cleaning & Feature Engineering	10%
EDA Depth & Creativity	20%
Prediction Accuracy (RMSE)	10%
Hyperparameter Tuning	10%
Deterministic Bid Sequence	15%

TASK 2: ADAPTIVE FITNESS & NUTRITION MAS (BONUS TASK)

Objective

Build a Multi-Agent System (MAS) that generates personalized, daily fitness and nutritional plans. The system must dynamically adapt its recommendations based on a simulated user's historical data, fatigue levels, goals, and physical limitations.

Suggested Tools & APIs

- LLM/Framework: LangChain, LangGraph, or CrewAI.
- Exercise Data: ExerciseDB API – Access to over 1,300 exercises with animated demonstrations.
- Nutrition Data: USDA FoodData Central API – For verifying macronutrients of suggested meals.
- Research: Tavily API – For fetching contextual health information and research tasks.
- How to use Gemini API

Some Ideas

- State Management (User History) - The architecture requires support for json or string-based inputs to ingest the user's real-time profile.
- Dynamic Workout Adjustment - Leveraging historical logs, the system must interface with external endpoints to construct injury-aware, customized training regimens.
- Nutritional Targeting - The system must suggest foods that align with the user’s goal
- Multi-Hop Tool use - If the system encounters a constraint outside its core database, it can use Tavily to research a solution and integrate it into the plan

Participants are encouraged to be creative in how they design the user state, the interaction between the agents, and the logic for adapting plans based on changing conditions. The evaluation focuses on your creative approach, the overall system robustness, and the efficacy with which your MAS navigates real-world physical constraints and evolving requirements.

Deliverables

1. Architecture Diagram: A flowchart showing system workflow and agent interaction (Dietitian, Coach, Orchestrator).
2. GitHub Repository: Public repository with complete code, requirements.txt, and setup instructions.
3. Video Demonstration: A maximum 3-minute recording showing the system reacting to a constraint change (e.g., adjusting a workout for a "shoulder injury").

DataSet Link

Task Submission - 28th May

Join the WhatsApp Group

Have any doubts ?
Contact us

Manthan +91 9512529995 Arnav +91 7982390939

Coding Club
IIT Guwahati